

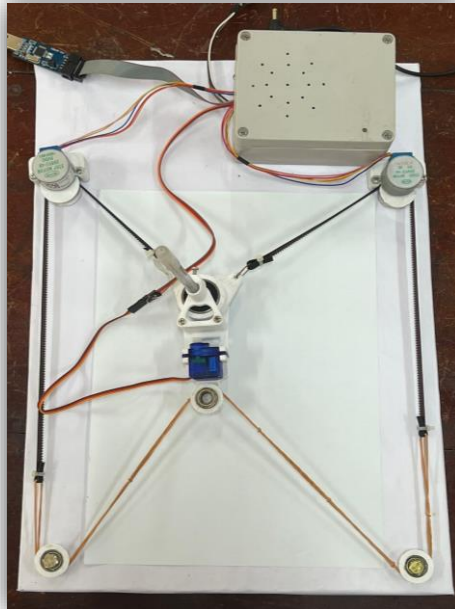
DEPARTMENT OF PHYSICS

UNIVERSITY OF COLOMBO

Embedded System Laboratory (PH3032)

2D Drawing Machine

By Serial Communication



Name: Thilanka Dissanayake

Index No: 16882

Registration No: 2022s19374

Date: September 15, 2025

1. Abstract.....	4
2. Introduction.....	5
2.1. Background and Motivation	5
2.2. Objectives.....	5
2.3. Scope and Limitations.....	6
3. Theoretical Principles	7
3.1. ATmega328P Microcontroller Architecture.....	7
3.1.1 Core and Memory	7
3.1.2 Peripherals.....	7
3.2 Stepper Motor Control	8
3.2.1 Principles of the 28BYJ-48 Motor	8
Figure 3.1. The stepper motor	8
3.2.2 ZC-A0591 Driver	8
3.2.3 Stepping Modes.....	8
3.3 Servo Motor Control.....	9
3.3.1 Principle of Operation	9
3.3.2 Pulse Width Modulation (PWM) on ATmega328P	9
3.4 Serial Communication Protocol (UART)	10
3.4.1 Asynchronous Data Transfer.....	10
3.4.2 Baud Rate Configuration	10
3.5 Motion Planning: Bresenham's Line Algorithm.....	11
3.5.1 Algorithm Overview.....	11
3.5.2 Implementation Logic	11
4. Methodology and Implementation.....	12
4.1 System Design Overview	12
4.2 Mechanical Design and Assembly	13
4.2.1 Cartesian Frame	13
Figure 4.1.	13
4.2.2 Pen Lift Mechanism	14
Figure 4.2. Operation of the servo motor	14
4.3 Hardware Design and Components.....	14
4.3.1 Hardware Components	14

4.3.2 Component Selection Rationale	15
4.4 PCB Design and Schematic	16
4.4.1 Schematic Description.....	16
Figure 4.1: Circuit Schematic	16
4.4.2 PCB Layout	17
Figure 4.2: PCB Layout	17
4.5 Firmware Development (C).....	18
4.5.1 Development Environment	18
4.5.2 Firmware Architecture.....	18
4.5.3 Peripheral Initialization.....	18
4.5.4 Command Processing.....	18
4.6 Control Software Development (Python).....	19
4.6.1 Software Architecture	19
4.6.2 High-Level Shape Functions	19
5. Results and Analysis	20
5.1 Experimental Setup.....	20
5.2 Qualitative Results	20
Figure 5.1: Sample output of Rectangle from the 2D Drawing Machine.	20
Figure 5.2: Sample output of Triangle from the 2D Drawing Machine.	20
Figure 5.3: Sample output of Star from the 2D Drawing Machine.	20
Figure 5.4: Sample output of Heart from the 2D Drawing Machine.....	20
Figure 5.5: Sample output of House from the 2D Drawing Machine.	21
5.3 Error Analysis	21
6. Discussion	22
6.1 Performance Evaluation	22
6.2 Challenges and Solutions.....	22
6.3 Future Improvements.....	23
7. Conclusion	24
8. Appendix.....	25
8.1. The C code	25
8.2. The Python code	34

1. Abstract

This report presents a detailed account of the design, development, and testing of a 2D Drawing Machine, a low-cost Cartesian plotter controlled by a personal computer. The primary objective of this project was to create an educational tool demonstrating core concepts in embedded systems, mechatronics, and automation. The system architecture is centered around an **ATmega328P microcontroller**, which orchestrates the movement of two **28BYJ-48 stepper motors** for X-Y axis control and a **SG90 servo motor** for actuating a pen-lift mechanism.

A custom-designed Printed Circuit Board (PCB) integrates the microcontroller with motor drivers and other peripheral components, ensuring a stable and compact hardware foundation. The mechanical structure is built using 3D-printed parts, bearings, and a belt-drive system to translate motor rotation into precise linear motion.

Communication between a host PC and the microcontroller is achieved via a **USB-TTL serial interface**, enabling the transmission of drawing commands. The firmware, written in C, implements a parser for a G-code-like command set and utilizes **Bresenham's line algorithm** for efficient, integer-based path interpolation. On the host side, a Python script provides a high-level interface for users, containing functions to draw various predefined geometric shapes and patterns by generating and sending the appropriate command sequences.

Performance tests demonstrate the machine's ability to successfully render complex shapes with reasonable accuracy. Analysis of the results identified potential sources of error, including mechanical backlash, belt tension inconsistencies, and motor step resolution. The project successfully meets its objectives, serving as a functional prototype and a robust platform for future enhancements in the field of hobbyist CNC technology.

2. Introduction

2.1. Background and Motivation

In the modern era of digital fabrication, Computer Numerical Control (CNC) has become a cornerstone technology, enabling the automated creation of complex parts with high precision. From industrial milling machines to desktop 3D printers, the principle remains the same: translating digital designs into physical objects through controlled motion. The 2D plotter is one of the earliest and most intuitive forms of CNC technology, providing a clear visual representation of digital path planning and motion control.

The motivation for this project stems from a desire to explore these foundational concepts in a hands-on, accessible manner. While commercial CNC machines are often expensive and complex, building a plotter from the ground up provides invaluable insight into the interplay between hardware, firmware, and software. This project, the "2D Drawing Machine," was conceived as an educational endeavor to design and implement a complete mechatronic system using readily available, low-cost components.

2.2. Objectives

The primary objectives of this project were defined as follows:

- **Design and build a functional Cartesian plotter** - capable of moving a pen in a 2D plane.
- **Develop a custom electronic control system** - based on the ATmega328P microcontroller, including the design and fabrication of a custom PCB.
- **Write robust firmware in C** - to manage motor control, interpret commands, and execute motion paths using an efficient algorithm.
- **Create a user-friendly Python control script** - for sending drawing commands from a PC to the plotter via serial communication.
- **Achieve reasonable drawing accuracy** - for a variety of predefined geometric shapes.
- **Document the entire process** - including design choices, challenges faced, and potential areas for improvement.

2.3. Scope and Limitations

The project's scope is focused on creating a proof-of-concept plotter. It is designed to draw single-color line art from a set of predefined functions. The machine operates on a small drawing area (approximately A5 paper size) and is not intended for high-speed or high-precision applications. Limitations include the inherent speed and torque constraints of the 28BYJ-48 stepper motors, the mechanical rigidity of the 3D-printed frame, and the simplicity of the G-code interpreter, which does not support advanced features like arc interpolation or buffered commands.

3. Theoretical Principles

3.1. ATmega328P Microcontroller Architecture

The ATmega328P is the central processing unit of the 2D Drawing Machine. It is a high-performance, low-power 8-bit microcontroller based on the AVR enhanced RISC architecture. Its versatility and extensive community support make it an ideal choice for this project

3.1.1 Core and Memory

The ATmega328P features 32 KB of In-System Programmable Flash memory for program storage, 2 KB of SRAM for data variables during runtime, and 1 KB of EEPROM for non-volatile data storage. The RISC architecture with 131 powerful instructions allows most instructions to execute in a single clock cycle, achieving throughputs approaching 1 MIPS per MHz.

3.1.2 Peripherals

The firmware for this project utilizes several of the ATmega328P's built-in peripherals:

- **General Purpose I/O (GPIO):**
The GPIO pins on PORTB and PORTD are configured as outputs to send the stepping signals to the ZC-A0591 motor drivers and the PWM signal to the servo motor.
- **Timers/Counters:**
The 16-bit Timer1 is used in Fast PWM mode to generate the precise 50 Hz PWM signal required for controlling the servo motor's angle. Its configuration involves setting the pre scaler and top value (ICR1) to define the period, and the compare match value (OCR1B) to define the duty cycle.
- **Universal Synchronous/Asynchronous Receiver/Transmitter (USART):**
The USART module is configured for asynchronous serial communication at a baud rate of 9600 bps to receive commands from the host PC.

3.2 Stepper Motor Control

3.2.1 Principles of the 28BYJ-48 Motor

The 28BYJ-48 is a 5-wire unipolar stepper motor. Internally, it has a step angle of 5.625° , meaning it takes $360/5.625=64$ steps to complete one full revolution of its rotor. However, it also includes an internal $1/64$ reduction gearbox. This results in an effective step count of $64 \times 64 = 4096$ steps per full revolution of the output shaft, providing very high resolution at the cost of speed.

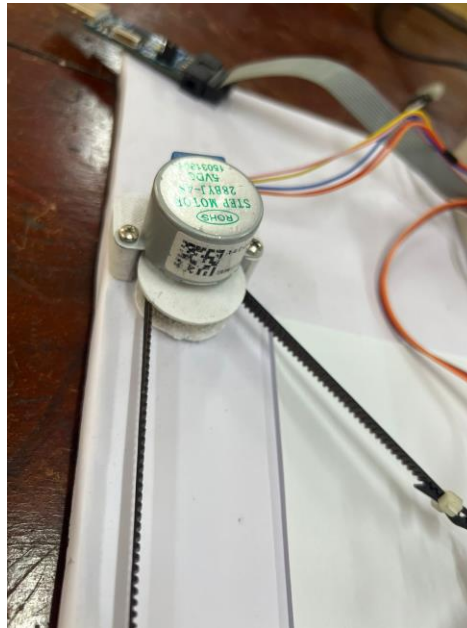


Figure 3.1. The stepper motor

3.2.2 ZC-A0591 Driver

The ATmega328P cannot directly provide the current required to drive the stepper motor coils. The ZC-A0591 driver board, which features a ULN2003 Darlington transistor array, acts as a current amplifier. It takes the low-current logic signals from the microcontroller's pins and switches the high-current path from the 5V power supply to the motor coils.

3.2.3 Stepping Modes

The firmware drives the motors using a half-step sequence. In this mode, both single coils and pairs of adjacent coils are energized sequentially. This doubles the motor's resolution (to 8 steps per cycle) and provides smoother rotation compared to full-stepping.

3.3 Servo Motor Control

3.3.1 Principle of Operation

The micro servo is a closed-loop control system. It consists of a DC motor, a gearbox, a position feedback potentiometer, and a control circuit. The control circuit receives a PWM signal and compares the corresponding target angle with the current angle read from the potentiometer. It then drives the DC motor until the difference between the two is zero, allowing for precise angular positioning.

3.3.2 Pulse Width Modulation (PWM) on ATmega328P

Standard servos operate on a PWM signal with a 20 ms (50 Hz) period. The width of the pulse within this period determines the angle:

- A ~1 ms pulse corresponds to 0°.
- A ~1.5 ms pulse corresponds to 90° (neutral).
- A ~2 ms pulse corresponds to 180°.

The firmware configures Timer1 in Fast PWM mode with ICR1 set to 19999 and a prescaler of 8. With an 8 MHz clock, this creates a 50 Hz period. The

OCR1B register is then set to a value between 1000 (1 ms) and 2000 (2 ms) to control the pen's angle.

3.4 Serial Communication Protocol (UART)

3.4.1 Asynchronous Data Transfer

UART is a form of asynchronous serial communication, meaning no clock signal is shared between the transmitter and receiver. Instead, the data is framed with a start bit and a stop bit to allow the receiver to synchronize. The data format used in this project is 8-N-1 (8 data bits, no parity bit, 1 stop bit), a common standard.

3.4.2 Baud Rate Configuration

Both the PC and the ATmega328P must be configured to operate at the same baud rate (9600 bps) for successful communication. In the firmware, this is achieved by loading a calculated value into the UBRR0 register. The formula used is

$$UBRR = (FCPU / (16 \times BAUD)) - 1$$

For an 8 MHz CPU frequency and 9600 baud, the UBRR value is approximately 51.

3.5 Motion Planning: Bresenham's Line Algorithm

3.5.1 Algorithm Overview

To draw a line from (x_0, y_0) to (x_1, y_1) , the plotter needs to execute a series of discrete steps in X and Y. Bresenham's algorithm is an optimal method for determining this sequence. It maintains an integer error term that represents the deviation of the current pixel grid point from the true mathematical line. At each iteration, it decides the next step (in X, Y, or diagonally) based on the sign of this error term, then updates the error term accordingly.

3.5.2 Implementation Logic

The `draw_line` function in the C firmware implements this algorithm. It first calculates the total steps required in X (`dx`) and Y (`dy`) and their directions (`sx`, `sy`). It then enters a loop that continues until both `steps_x` and `steps_y` counters reach zero. Inside the loop, an error term `err` is compared. If the error term indicates that the path is drifting too far in one direction, a corrective step is made in the other direction. This ensures the plotted line remains as close as possible to the ideal straight line, using only efficient integer arithmetic.

4. Methodology and Implementation

4.1 System Design Overview

The system is divided into three main subsystems:

1. **Mechanical System:** The physical plotter frame, belts, and pen holder.
2. **Electronic System:** The custom PCB with the ATmega328P, motor drivers, and power supply.
3. **Software System:** The C firmware on the microcontroller and the Python control script on the PC.

A user interacts with the Python script, which sends high-level commands. The firmware receives these commands, performs low-level motion planning, and drives the motors to create the final drawing.

4.2 Mechanical Design and Assembly

4.2.1 Cartesian Frame

The machine's structure was built primarily from 3D-printed parts, designed to be lightweight and easy to assemble. The design follows a CoreXY-like belt arrangement where the position of the central carriage is determined by the coordinated movement of both stepper motors. This design minimizes the moving mass, as the motors themselves are stationary. Copper-coated anti-slip belts are used to transmit motion from the motor pulleys to the pen carriage, with bearings serving as idler pulleys to maintain belt tension and ensure smooth motion.



Figure 4.1.

4.2.2 Pen Lift Mechanism

The pen is held in a 3D-printed holder attached to the central carriage. A micro servo motor is mounted on this carriage, with its horn connected to a lever that lifts the entire pen holder assembly up and down. This simple yet effective mechanism allows the machine to move between points without drawing on the paper



Figure 4.2. Operation of the servo motor

4.3 Hardware Design and Components

4.3.1 Hardware Components

- 1 ATmega328P-PU Microcontroller
- 2 28BYJ-48 Stepper Motors with ZC-A0591 Drivers
- 1 Micro Servo Motor SG90
- 1 USB-TTL CH340 Adapter for Serial Communication
- 1 5V 2A DC Power Adapter
- 3 (8-16mm) Bearings and 2 (12-28mm) Bearings
- 1 Custom designed single-layer PCB with Required Components|
- 3D Printed Parts & belts

4.3.2 Component Selection Rationale

- **ATmega328P:** Chosen for its low cost, wide availability, extensive documentation, and sufficient peripherals for this application.
- **28BYJ-48 Steppers:** Selected due to their very low cost and high resolution, making them suitable for a budget-friendly, proof-of-concept plotter where speed is not a primary concern.
- **Micro Servo:** A standard, inexpensive solution for simple actuation tasks like the pen lift.
- **5V 2A Power Supply:** Chosen to provide adequate current for both stepper motors (which can draw several hundred mA each) and the servo, ensuring stable operation.

4.4 PCB Design and Schematic

4.4.1 Schematic Description

The schematic, designed in EasyEDA, outlines all electrical connections. The ATmega328P is the central component. The schematic includes a 16 MHz crystal oscillator circuit, a reset button with a pull-up resistor, and headers for connecting the two stepper drivers (8 pins total), the servo (3 pins), and the USB-TTL adapter (for power, ground, RX, TX). The motor drivers are powered directly from the main 5V input, while the ATmega328P's VCC is also connected to this rail, potentially through a filtering inductor and capacitor to reduce motor noise.

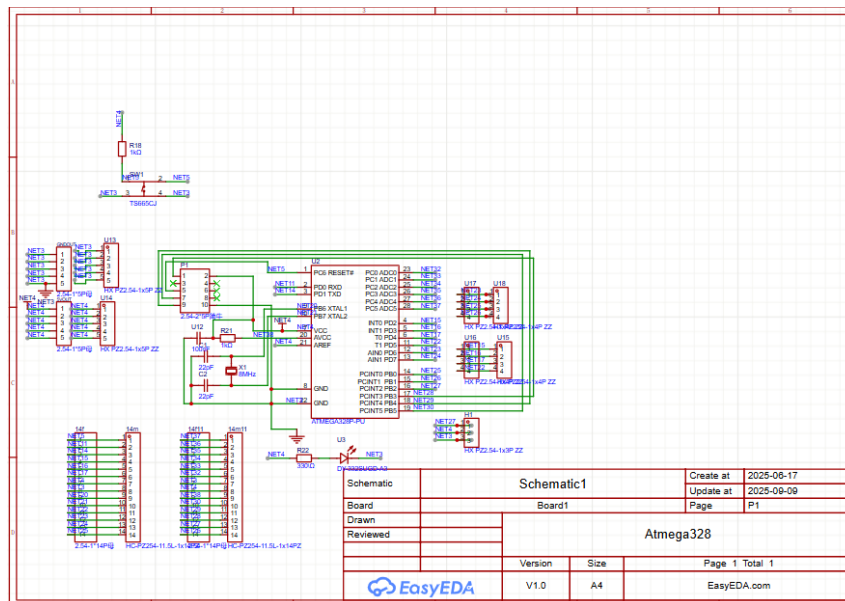


Figure 4.1: Circuit Schematic

4.4.2 PCB Layout

The PCB is a single-sided board designed for easy home fabrication. Component placement was optimized to keep related components close. Traces for the motor outputs are made wider to handle higher currents. A ground plane is used to improve signal integrity and reduce electrical noise generated by the motors. Headers are grouped logically for easy connection of the external modules.

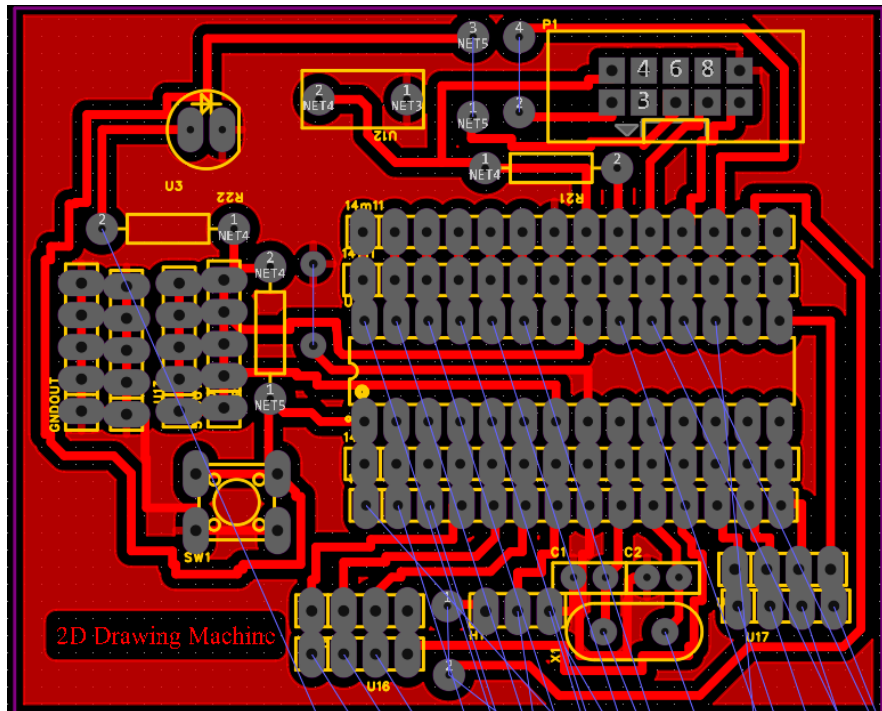


Figure 4.2: PCB Layout

4.5 Firmware Development (C)

4.5.1 Development Environment

The firmware was written in C using an AVR toolchain (avr-gcc, avr-libc) and programmed onto the ATmega328P using an external programmer via the ICSP header on the PCB.

4.5.2 Firmware Architecture

The firmware's architecture is based on a simple super-loop. After an initial setup phase, the `main` function enters an infinite `while(1)` loop. Inside this loop, it continuously checks for incoming serial data. When a newline character is received, the assembled command string is passed to the `process_gcode_command` function for parsing and execution. This is a blocking, single-tasking architecture, sufficient for the requirements of this project.

4.5.3 Peripheral Initialization

The `setup_pins()` function configures the data direction registers (DDRB, DDRD) for all motor control pins as outputs. The `USART_Init()` function configures the baud rate and enables the UART transmitter and receiver. The `timerservo_init()` function configures `Timer1` for Fast PWM mode to drive the servo.

4.5.4 Command Processing

The `process_gcode_command` function acts as the core of the firmware's logic. It uses `strcmp` and `strncmp` to identify commands:

- **M3/M5:** Directly call `servo_set_angle` to lift or lower the pen.
- **G28 (Home):** Calculate the steps needed to move from the `current_x`, `current_y` position back to (0,0) and execute the movement.
- **G1 (Linear Move):** Parses the command string using `atoi` to extract the target X and Y coordinates. It then calculates the delta steps required and calls `execute_movement`, which in turn calls `draw_line`. Finally, it updates the global `current_x` and `current_y` variables.

4.6 Control Software Development (Python)

4.6.1 Software Architecture

The Python script is object-oriented, with a `DrawingRobot` class that encapsulates all communication and control logic. Upon instantiation, the class constructor initializes the serial connection via the `pyserial` library and waits for a "Ready" message from the firmware, confirming the connection is live. The class provides methods like `move_to`, `pen_up`, and `pen_down` that abstract the underlying G-code commands.

4.6.2 High-Level Shape Functions

The script includes functions like `draw_square`, `draw_triangle`, and `draw_heart`. Each of these functions contains the geometric logic to generate a series of (X, Y) waypoints for that shape. It then calls the basic

`move_to`, `pen_down`, and `pen_up` methods in the correct sequence to draw the shape on the machine. For example, `draw_square` calculates the coordinates of the four corners and sends four `G1` commands between them. The `draw_heart` function uses a parametric equation to generate a dense set of points to approximate a smooth curve.

5. Results and Analysis

5.1 Experimental Setup

The machine was placed on a flat, stable surface. An A5 sheet of paper was secured to the drawing platform. The machine was powered on with a 5V 2A DC adapter, and the USB-TTL adapter was connected to a host PC running the Python control script. The

`draw_shapes.py` script was executed from the command line, specifying the correct serial port.

5.2 Qualitative Results

The machine was commanded to draw a sequence of shapes, including a square, star, triangle, heart, and letters. The output was visually inspected. The machine successfully drew all shapes, demonstrating that the end-to-end system was functional. The lines were continuous, and the pen lift mechanism worked reliably between separate drawing paths.



Figure 5.1: Sample output of Rectangle from the 2D Drawing Machine.



Figure 5.2: Sample output of Triangle from the 2D Drawing Machine.



Figure 5.3: Sample output of Star from the 2D Drawing Machine.



Figure 5.4: Sample output of Heart from the 2D Drawing Machine.



Figure 5.5: Sample output of House from the 2D Drawing Machine.

The drawings are clearly recognizable. However, some minor artifacts are visible upon close inspection, such as slight waviness in straight lines, Pen lift issues and rounded corners, which are analyzed below.

5.3 Error Analysis

Several sources contribute to the observed inaccuracies:

- **Mechanical Backlash:** There is inherent play in the gearboxes of the 28BYJ-48 motors and slack in the belt system. This causes a small amount of lost motion when the direction of movement changes, leading to rounded corners and slight inaccuracies.
- **Frame Rigidity:** The 3D-printed frame can flex slightly during rapid movements, causing vibrations that manifest as wavy lines.
- **Motor Step Loss:** If the pen experiences too much friction or the machine attempts to move too fast, the motors may not have enough torque to complete a step, causing the machine to lose its position. This is mentioned as a potential issue in the user manual's troubleshooting section.
- **Belt Tension:** Uneven or inadequate belt tension can lead to slipping or stretching, resulting in dimensional inaccuracies and skewing of the drawing.
- **Pen Tip Friction:** The friction between the pen tip and the paper creates a drag force that the motors must overcome. Variations in this friction can affect positioning accuracy.
- **Servo Motor Torque Limitations:** The SG90 micro servo motor is lightweight and inexpensive, but it has limited power. When used to lift the pen, the total weight of the pen holder, the pen itself, and especially the 12-28mm bearings in the assembly can exceed the motor's torque capacity.

6. Discussion

6.1 Performance Evaluation

The 2D Drawing Machine successfully meets all the primary objectives outlined for the project. It demonstrates a complete, functional mechatronic system capable of translating digital commands into physical drawings. The modular design, combining a custom PCB with 3D-printed parts and a Python control interface, proved to be an effective and flexible approach. The firmware's implementation of Bresenham's algorithm is efficient and the G-code-like protocol is robust enough for the intended tasks. The project serves as an excellent educational platform for learning about CNC principles in a practical setting.

6.2 Challenges and Solutions

- **Challenge:** Initial motor movements were jerky and inaccurate.
 - **Solution:** The issue was traced to the stepping mode. Switching from a full-step to a half-step sequence in the firmware (`stepper_sequence` array) resulted in much smoother motion.
- **Challenge:** The machine would occasionally lose its position during longer drawings.
 - **Solution:** This was identified as motor step loss due to insufficient power. The troubleshooting guide in the user manual suggests verifying the 5V 2A power supply is used. Additionally, the `_delay_ms(1)` in the `step_motor` function was crucial to give the motors time to complete each step, reducing the chance of stalls.
- **Challenge:** Calibrating the drawing size to match real-world units.
 - **Solution:** The `UNIT_STEPS` constant in the firmware required significant trial-and-error adjustment. A calibration procedure was developed where a line of known length was commanded, and the constant was adjusted until the drawn length matched the expected length.

6.3 Future Improvements

While the project was successful, there are numerous avenues for enhancement:

- **Upgrade Drive System:** Replace the 28BYJ-48 motors with NEMA 17 stepper motors and A4988 drivers. This would dramatically increase speed, torque, and accuracy.
- **Improve Mechanical Design:** Utilize linear rails or metal rods instead of a purely 3D-printed guide system to increase rigidity and reduce vibrations.
- **Advanced Firmware:** Implement a command buffer (circular buffer) in the firmware to store multiple G-code commands. This would allow for look-ahead planning and acceleration/deceleration control, resulting in smoother and faster movements, especially for curves.
- **Implement Closed-Loop Control:** Add rotary encoders to the motors to provide positional feedback, allowing the firmware to detect and correct for lost steps.
- **Develop a Graphical User Interface (GUI):** Create a desktop application using a framework like PyQt or Tkinter that allows users to load, preview, and draw standard vector graphics files (e.g., SVG), rather than being limited to hard-coded shapes.
- **Text Writing:** Create python script would have pre-defined G-code templates for each letter, which it would then execute one by one to form words and sentences. This would transform the machine into a simple text-writing robot.
- **Image-to-G-code Conversion:** Develop a software pipeline that takes a simple black-and-white image as input and converts it into a series of G-code commands. This process would involve edge detection or image tracing to identify the lines and contours, which are then translated into linear (G1) move commands. The Python script could then read and send these commands to the machine, allowing it to "sketch" simple pictures.

7. Conclusion

The Design and Implementation of a PC-Controlled 2D Drawing Machine project successfully resulted in the creation of a functional and educational Cartesian plotter. By integrating an ATmega328P microcontroller, stepper and servo motors, a custom PCB, and a Python-based control interface, the project comprehensively covered key aspects of embedded systems engineering and mechatronics. The machine demonstrated its capability to draw various geometric shapes with a degree of accuracy suitable for its low-cost, hobbyist nature.

The development process provided critical insights into the challenges of motion control, including mechanical precision, motor limitations, and the importance of robust firmware. The final prototype, while having scope for improvement, stands as a testament to the power of integrating simple, accessible components to create a complex automated system. It not only fulfills all its initial objectives but also serves as a versatile platform for future exploration and enhancement in the field of digital fabrication.

8. Appendix

8.1. The C code

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <stdlib.h>
#include <util/delay.h>
#include <stdint.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include <string.h>

// --- Serial Communication ---
#define BAUD 9600
#define MY_UBRR F_CPU/16/BAUD-1

// --- Stepper Motor Pin Definitions ---
#define X_STEPPER_PORT PORTD
#define X_STEPPER_DDR DDRD
#define X_STEPPER_IN1_PIN PD2
#define X_STEPPER_IN2_PIN PD3
#define X_STEPPER_IN3_PIN PD4
#define X_STEPPER_IN4_PIN PD5

#define Y_STEPPER_PORT_D PORTD
#define Y_STEPPER_DDR_D DDRD
#define Y_STEPPER_IN1_PIN PD6
#define Y_STEPPER_IN2_PIN PD7

#define Y_STEPPER_PORT_B PORTB
#define Y_STEPPER_DDR_B DDRB
#define Y_STEPPER_IN3_PIN PB0
#define Y_STEPPER_IN4_PIN PB1

// --- Servo Pin Definition ---
#define SERVO_PIN PB2
```

```

// --- Calibration Constant ---
#define UNIT_STEPS 500
#define MAX_X 1000 // Maximum X coordinate
#define MAX_Y 1000 // Maximum Y coordinate

// Stepper sequence
const uint8_t stepper_sequence[] = {
    0b0001, 0b0011, 0b0010, 0b0110, 0b0100, 0b1100, 0b1000, 0b1001
};

// --- Function Prototypes ---
void USART_Init(unsigned int ubrr);
void USART_Transmit(unsigned char data);
unsigned char USART_Receive(void);
void USART_PrintString(const char *str);
void setup_pins();
void step_motor(int motor_x, int motor_y, int steps_x, int steps_y);
void draw_line(int steps_x, int steps_y);
void timerservo_init(void);
void servo_set_angle(uint32_t angle);
void process_gcode_command(char *command);
void execute_movement(int x_steps, int y_steps);
void draw_diamond_pattern(void); // New function for diamond pattern

// Global variables
int current_x = 0;
int current_y = 0;
int servo_angle = 0;

int main(void) {
    // Initialize peripherals
    USART_Init(MY_UBRR);
    setup_pins();
    timerservo_init();
    servo_set_angle(10); // Start with pen up
}

```

```

// Send ready message
USART_PrintString("Ready\n");

char buffer[32];
uint8_t index = 0;

while (1) {
    // Read serial data
    unsigned char received = USART_Receive();

    if (received == '\n' || received == '\r') {
        if (index > 0) {
            buffer[index] = '\0'; // Null terminate
            process_gcode_command(buffer);
            index = 0;
        }
        } else if (index < sizeof(buffer) - 1) {
            buffer[index++] = received;
        }
    }
    return 0;
}

// --- Serial Communication Functions ---
void USART_Init(unsigned int ubrr) {
    UBRR0H = (unsigned char)(ubrr >> 8);
    UBRR0L = (unsigned char)ubrr;
    UCSR0B = (1 << RXEN0) | (1 << TXEN0);
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}

void USART_Transmit(unsigned char data) {
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = data;
}

```

```

unsigned char USART_Receive(void) {
    while (!(UCSR0A & (1 << RXC0)));
    return UDR0;
}

void USART_PrintString(const char *str) {
    while (*str) {
        USART_Transmit(*str++);
    }
}

// --- G-code Processing ---
void process_gcode_command(char *command) {
    // Simple G-code parser
    if (strcmp(command, "M3") == 0) {
        // Pen down
        servo_set_angle(160);
        servo_angle = 160;
        USART_PrintString("OK\n");
    }
    else if (strcmp(command, "M5") == 0) {
        // Pen up
        servo_set_angle(10);
        servo_angle = 10;
        USART_PrintString("OK\n");
    }
    else if (strcmp(command, "G28") == 0) {
        // Home command - return to origin
        execute_movement(-current_x, -current_y);
        current_x = 0;
        current_y = 0;
        USART_PrintString("OK\n");
    }
    else if (strcmp(command, "DIAMOND") == 0) {
        // Draw diamond pattern using the four corners
        draw_diamond_pattern();
        USART_PrintString("OK\n");
    }
}

```

```

else if (strncmp(command, "G1", 2) == 0) {
    // G1 movement command: G1 X100 Y200
    char *ptr = command;
    int x_target = current_x;
    int y_target = current_y;

    // Parse coordinates
    while (*ptr) {
        if (*ptr == 'X' || *ptr == 'x') {
            x_target = atoi(ptr + 1);
        } else if (*ptr == 'Y' || *ptr == 'y') {
            y_target = atoi(ptr + 1);
        }
        ptr++;
    }

    // Boundary check
    if (x_target > MAX_X) x_target = MAX_X;
    if (x_target < -MAX_X) x_target = -MAX_X;
    if (y_target > MAX_Y) y_target = MAX_Y;
    if (y_target < -MAX_Y) y_target = -MAX_Y;

    // Calculate steps to move
    int x_steps = x_target - current_x;
    int y_steps = y_target - current_y;

    // Execute movement
    execute_movement(x_steps, y_steps);

    // Update current position
    current_x = x_target;
    current_y = y_target;

    USART_PrintString("OK\n");
}

```

```

    else {
        USART_PrintfString("ERR: Unknown command\n");
    }
}

// New function to draw diamond pattern using four corners
void draw_diamond_pattern(void) {
    // South West corner
    execute_movement(-MAX_X, -MAX_Y);
    current_x = -MAX_X;
    current_y = -MAX_Y;
    _delay_ms(500);

    // North West corner
    execute_movement(0, MAX_Y * 2);
    current_x = -MAX_X;
    current_y = MAX_Y;
    _delay_ms(500);

    // North East corner
    execute_movement(MAX_X * 2, 0);
    current_x = MAX_X;
    current_y = MAX_Y;
    _delay_ms(500);

    // South East corner
    execute_movement(0, -MAX_Y * 2);
    current_x = MAX_X;
    current_y = -MAX_Y;
    _delay_ms(500);

    // Back to South West to complete diamond
    execute_movement(-MAX_X * 2, 0);
    current_x = -MAX_X;
    current_y = -MAX_Y;
    _delay_ms(500);
}

```

```

    // Return to center
    execute_movement(MAX_X, MAX_Y);
    current_x = 0;
    current_y = 0;
}

void execute_movement(int x_steps, int y_steps) {
    if (x_steps != 0 || y_steps != 0) {
        draw_line(x_steps, y_steps);
        _delay_ms(100);
    }
}

// --- Hardware Control Functions ---
void setup_pins() {
    // Stepper pins
    X_STEPPER_DDR |= (1 << X_STEPPER_IN1_PIN) | (1 << X_STEPPER_IN2_PIN) |
                    (1 << X_STEPPER_IN3_PIN) | (1 << X_STEPPER_IN4_PIN);
    Y_STEPPER_DDR_D |= (1 << Y_STEPPER_IN1_PIN) | (1 << Y_STEPPER_IN2_PIN);
    Y_STEPPER_DDR_B |= (1 << Y_STEPPER_IN3_PIN) | (1 << Y_STEPPER_IN4_PIN);

    // Servo pin
    DDRB |= (1 << SERVO_PIN);
}

void timerservo_init(void) {
    TCCR1A = (1 << COM1B1) | (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11);
    ICR1 = 19999;
}

void servo_set_angle(uint32_t angle) {
    uint16_t ocr_value = 1000 + (angle * 1000) / 180;
    OCR1B = ocr_value;
    _delay_ms(15);
}

```

```

void step_motor(int motor_x, int motor_y, int steps_x, int steps_y) {
    static int sequence_index_x = 0;
    static int sequence_index_y = 0;

    if (motor_x) {
        X_STEPPER_PORT &= ~((1 << X_STEPPER_IN1_PIN) | (1 << X_STEPPER_IN2_PIN) |
                               (1 << X_STEPPER_IN3_PIN) | (1 << X_STEPPER_IN4_PIN));
        sequence_index_x = (sequence_index_x + steps_x + 8) % 8;
        uint8_t current_step = stepper_sequence[sequence_index_x];
        if (current_step & (1 << 0)) X_STEPPER_PORT |= (1 << X_STEPPER_IN1_PIN);
        if (current_step & (1 << 1)) X_STEPPER_PORT |= (1 << X_STEPPER_IN2_PIN);
        if (current_step & (1 << 2)) X_STEPPER_PORT |= (1 << X_STEPPER_IN3_PIN);
        if (current_step & (1 << 3)) X_STEPPER_PORT |= (1 << X_STEPPER_IN4_PIN);
    }

    if (motor_y) {
        Y_STEPPER_PORT_D &= ~((1 << Y_STEPPER_IN1_PIN) | (1 << Y_STEPPER_IN2_PIN));
        Y_STEPPER_PORT_B &= ~((1 << Y_STEPPER_IN3_PIN) | (1 << Y_STEPPER_IN4_PIN));
        sequence_index_y = (sequence_index_y + steps_y + 8) % 8;
        uint8_t current_step = stepper_sequence[sequence_index_y];
        if (current_step & (1 << 0)) Y_STEPPER_PORT_D |= (1 << Y_STEPPER_IN1_PIN);
        if (current_step & (1 << 1)) Y_STEPPER_PORT_D |= (1 << Y_STEPPER_IN2_PIN);
        if (current_step & (1 << 2)) Y_STEPPER_PORT_B |= (1 << Y_STEPPER_IN3_PIN);
        if (current_step & (1 << 3)) Y_STEPPER_PORT_B |= (1 << Y_STEPPER_IN4_PIN);
    }
    _delay_ms(1);
}

void draw_line(int steps_x, int steps_y) {
    if (steps_x == 0 && steps_y == 0) return;

    int dx = abs(steps_x);
    int dy = abs(steps_y);
    int sx = (steps_x > 0) ? 1 : -1;
    int sy = (steps_y > 0) ? 1 : -1;
    int err = dx - dy;

```



```
while (1) {  
    if (steps_x == 0 && steps_y == 0) break;  
    int e2 = 2 * err;  
  
    if (e2 > -dy) {  
        err -= dy;  
        step_motor(1, 0, sx, 0);  
        steps_x -= sx;  
    }  
    if (e2 < dx) {  
        err += dx;  
        step_motor(0, 1, 0, sy);  
        steps_y -= sy;  
    }  
}
```

8.2. The Python code

```
import serial
import time
import sys
import math

class DrawingRobot:
    def __init__(self, serial_port):
        self.ser = serial.Serial(serial_port, 9600, timeout=2)
        time.sleep(2)
        self.ser.reset_input_buffer()
        print("Waiting for microcontroller...")

        # Wait for ready message
        ready_message = self.ser.readline().decode('utf-8', errors='ignore').strip()
        if ready_message == "Ready":
            print("Microcontroller is ready!")
        else:
            print("Connection established, but no ready message received.")

    def send_command(self, command):
        """Send a command and wait for response"""
        print(f">>> Sending: {command}")
        self.ser.write((command + "\n").encode('utf-8'))

        while True:
            response = self.ser.readline().decode('utf-8', errors='ignore').strip()
            if response:
                print(f"<<< Received: {response}")
                if response == "OK" or response.startswith("ERR"):
                    return response
            time.sleep(0.1)

    def pen_up(self):
        """Lift the pen"""
        return self.send_command("M3")
```

```

def pen_down(self):
    """Lower the pen"""
    return self.send_command("M5")

def move_to(self, x, y):
    """Move to absolute coordinates"""
    return self.send_command(f"G1 X{x} Y{y}")

def home(self):
    """Return to origin (0,0)"""
    return self.send_command("G28")

def draw_diamond(self):
    """Draw diamond using the built-in DIAMOND command"""
    self.pen_down()
    self.send_command("DIAMOND")
    self.pen_up()

def draw_square(self, size=10):
    """Draw a square centered at origin"""
    half_size = size // 2
    self.pen_down()
    self.move_to(-half_size, -half_size) # Bottom left
    self.move_to(-half_size, half_size) # Top left
    self.move_to(half_size, half_size) # Top right
    self.move_to(half_size, -half_size) # Bottom right
    self.move_to(-half_size, -half_size) # Back to start
    self.pen_up()

```

```

def draw_rectangle(self, width=1000, height=600):
    """Draw a rectangle centered at origin"""
    half_w = width // 2
    half_h = height // 2
    self.pen_down()
    self.move_to(-half_w, -half_h) # Bottom left
    self.move_to(-half_w, half_h) # Top left
    self.move_to(half_w, half_h) # Top right
    self.move_to(half_w, -half_h) # Bottom right
    self.move_to(-half_w, -half_h) # Back to start
    self.pen_up()

def draw_triangle(self, size=800):
    """Draw an equilateral triangle"""
    height = int(size * math.sqrt(3) / 2)
    self.pen_down()
    self.move_to(0, -size//2) # Bottom center
    self.move_to(-size//2, height//2) # Left vertex
    self.move_to(size//2, height//2) # Right vertex
    self.move_to(0, -size//2) # Back to bottom
    self.pen_up()

def draw_cross(self, size=800):
    """Draw a plus sign/cross"""
    half_size = size // 2
    self.pen_down()
    self.move_to(0, -half_size) # Bottom
    self.move_to(0, half_size) # Top
    self.pen_up()
    self.move_to(-half_size, 0) # Move to left
    self.pen_down()
    self.move_to(half_size, 0) # Right
    self.pen_up()

```

```

def draw_house(self, size=600):
    """Draw a simple house shape"""
    half_size = size // 2
    roof_height = int(size * 0.4)

    self.pen_down()
    # House base
    self.move_to(-half_size, -half_size) # Bottom left
    self.move_to(-half_size, half_size) # Top left
    self.move_to(half_size, half_size) # Top right
    self.move_to(half_size, -half_size) # Bottom right
    self.move_to(-half_size, -half_size) # Back to start

    # Roof
    self.move_to(-half_size, half_size) # Top left
    self.move_to(0, half_size + roof_height) # Roof peak
    self.move_to(half_size, half_size) # Top right

    self.pen_up()

def draw_star(self, size=600, points=5):
    """Draw a star with specified points"""
    self.pen_down()
    for i in range(points * 2):
        angle = math.pi * i / points
        radius = size // 2 if i % 2 == 0 else size // 4
        x = int(radius * math.sin(angle))
        y = int(radius * math.cos(angle))
        if i == 0:
            self.move_to(x, y)
        else:
            self.move_to(x, y)
    self.pen_up()

```

```

def draw_letter_T(self, size=600):
    """Draw the letter T"""
    half_size = size // 2
    self.pen_down()
    # Top horizontal bar
    self.move_to(-half_size, half_size)
    self.move_to(half_size, half_size)
    # Vertical stem
    self.move_to(0, half_size)
    self.move_to(0, -half_size)
    self.pen_up()

def draw_heart(self, size=400):
    """Draw a heart shape"""
    points = []
    for t in range(0, 628, 10): # 0 to 2π in steps of 0.1
        t_rad = t / 100
        x = 16 * math.sin(t_rad) ** 3
        y = 13 * math.cos(t_rad) - 5 * math.cos(2*t_rad) - 2 * math.cos(3*t_rad) - math.cos(4*t_rad)
        points.append((int(x * size/16), int(-y * size/16)))

    self.pen_down()
    for x, y in points:
        self.move_to(x, y)
    self.pen_up()

def close(self):
    """Close the serial connection"""
    self.ser.close()

def main():
    if len(sys.argv) != 2:
        print("Usage: python3 draw_shapes.py <serial_port>")
        print("Example: python3 draw_shapes.py /dev/ttyUSB0")
        sys.exit(1)

```

```

serial_port = sys.argv[1]

try:
    robot = DrawingRobot(serial_port)

    print("\n=== Drawing Demo ===\n")

    # Draw various shapes
    shapes = [
        ("Square", lambda: robot.draw_square(700)),
        ("Rectangle", lambda: robot.draw_rectangle(900, 500)),
        ("Triangle", lambda: robot.draw_triangle(700)),
        ("Cross", lambda: robot.draw_cross(600)),
        ("House", lambda: robot.draw_house(500)),
        ("Star", lambda: robot.draw_star(500, 5)),
        ("Letter T", lambda: robot.draw_letter_T(500)),
        ("Heart", lambda: robot.draw_heart(300)),
        ("Diamond", lambda: robot.draw_diamond()),
    ]

    for shape_name, shape_func in shapes:
        print(f"\nDrawing {shape_name}...")
        shape_func()
        robot.home()
        time.sleep(5)

    print("\n=== Drawing complete! ===")
    robot.close()

except serial.SerialException as e:
    print(f"Error: Could not open serial port {serial_port}.")
    print(f"Details: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    main()

```